Cloud Computing

Tutorial 3 Cloud Security and Information Retrieval

1) The rapid advancement of Generative AI demands a reassessment of crowdsourcing's traditional role. Begin by analyzing cloud platforms like AWS Mechanical Turk (https://www.mturk.com)-used to create ImageNet and Google's Crowdsource (https://crowdsource.google.com), which harnesses micro-tasks to improve core services. How did these human-in-the-loop systems create value that other approaches could not, and why were they essential to the first wave of cloud computing?

   Think how to evolve the reCAPTCHA principle in the Gen-AI era and explain how your solution creates a "human signal" to solving a genuine problem.

2) Using Diffie-Hellman key exchange, Alice and Bob agree to use p=23 and g=5 (publicly). Alice secretly uses a=6, and Bob secretly chooses b=13. What is their common secret key?

3) Alice wants to sign a message using the RSA digital signature scheme to ensure authenticity and integrity. She selects the following RSA parameters:

   Two prime numbers: p=47, q=59
   Public exponent: e=17

   i) Compute the modulus n and Euler's totient function $\phi(n)$. Determine the private key exponent d such that d is the modular inverse of e modulo $\phi(n)$.
   ii) Alice wants to sign the message M=42. Compute the digital signature S using her private key. Bob receives the message M and signature S. Explain how he verifies the authenticity of Alice's signature using her public key.
   iii) Discuss why digital signatures provide both authentication and non-repudiation in cryptographic communication.

4) Fuzzing—testing unexpected input variations—can reveal bypasses in cloud computing services and a recent example is Microsoft Fuzzing-as-a-Service (https://www.microsoft.com/en-us/research/blog/a-brief-introduction-to-fuzzing-and-why-its-an-important-tool-for-developers/). SaaS applications can use input filters to block attacks. Here, we analyze manually fuzzing a cloud-based login filter to reveal this vulnerability. Its API gateway filter blocks passwords containing:

   = (prevents cloud parameter pollution)

   ' (prevents NoSQL injection)

   DROP (prevents database tampering)

   But this API gateway filter only performs exact, case-sensitive matching.

   (i) Generate **10 fuzzed password inputs** designed to bypass this cloud filter.

   Start with "studentpassword" and create two examples for each technique:

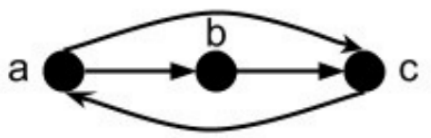
Technique	Your Fuzzed Inputs (2 each)
Case variation	1. "studentPassword" 2.

Technique	Your Fuzzed Inputs (2 each)	
Character substitution	1.	2.
URL encoding (e.g., %27 for ')	1.	2.
Character padding	1.	2.
Combination	1.	2.

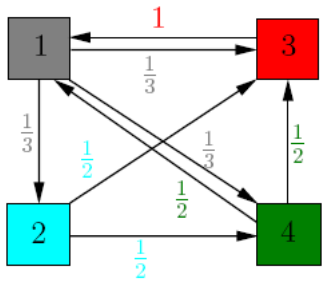
(ii) Predict the Bypass Success Rate Percentage.

(iii) Now consider how *crowdsourcing could be weaponized* to enhance your fuzzing attack against the SaaS platform. Imagine the attacker distributes your fuzzing task to a global network of 500 anonymous workers through a crowdsourcing platform (e.g., AWS MTurk), each paid a small fee to manually try different password variations on the login page. Explain how this *crowdsourced fuzzing attack* differs from automated fuzzing in terms of evasion rate, considering that human workers can naturally adapt to context and test specific variations that automated tools might miss. What are possible defense against *crowdsourced fuzzing attack*?

5) Compute the PageRank for the following two web graphs. Assume that the random web surfer jumps off (i.e., teleport) to another webpage from the current webpage that the web surfer is viewing with probability 0.15.



(a)



(b)

6) Consider a web graph with Google matrix M where a random web surfer jumps off (i.e., "teleport") to another website (with all webpages equally likely) from the current website with probability p. Derive the PageRank in analytical form (i.e., in closed form) in terms of M and p. Describe how the PageRank may change with respect to p. Explain using illustrative examples.

Implement the Pagerank algorithm using your favorite programming language (e.g., Python, C++, see link below) and evaluated on large datasets, e.g., web-Google_10k.txt or web-Google.txt provided by Google in 2002 available at:

https://hunglvosu.github.io/posts/2020/07/PA3/. You can use any graph processing libraries (see references) or develop your own in your program. Compare the numerical output of your algorithm with the above closed form.

More Challenging: Next, you will explore PageRank's modern relevance by choosing either: (i) AI web crawling for training or otherwise (ii) GraphRAG for information retrieval. You can choose to use or create any software tools.

(i) OpenAI uses specialized web crawlers like GPTBot (https://developers.openai.com/api/docs/bots/) to collect training data for foundation models with website owners able to control access via Web Robot configuration (https://www.robotstxt.org/). Imagine you are designing a crawling strategy for a new AI research organization and must decide how to prioritize which newly-discovered web pages to crawl first using a GPTBot-like crawler. Write a program/software that takes a small directed web graph (represented as a dictionary of URLs and their outlinks) and precomputed PageRank scores, then returns the top k URLs to crawl based on authority, explaining why high-PageRank pages might yield higher-quality training data for generative AI models. Propose one heuristic to find high-quality pages that permit crawling.

(ii) Explore the open-source repository fast-graphrag by circlemind-ai on (https://github.com/circlemind-ai/fast-graphrag), which is a GraphRAG framework for interpretable agent-driven retrieval workflows. Its primary goal is to support for dynamic, incrementally updatable knowledge graphs by PageRank to enhance accuracy and dependability. Use GraphRAG to answer a multi-hop query, for instance: "What discoveries by Marie Curie led to later advances in medical imaging?" Injecting Pagerank scores into the retrieval step helps by starting from seed entities ("Marie Curie," "medical imaging") and propagating importance through the graph's connections (e.g., "Marie Curie" -> "discovered radium" -> "used in radiotherapy" -> "advances in medical imaging"), effectively ranking nodes based on their relevance to the specific query context. Write a program/software to take a knowledge graph and a set of query entities to return the top k most relevant nodes.

References:

- Pagerank in Python sample code: https://github.com/Shesh6/CS166--Modeling-Simulation-and-Decision-Making/blob/master/Homework%2082%20-%20PageRank%20as%20a%20Random%20Walk.ipynb

- Pagerank in C/Java sample code: https://github.com/GraphChi
https://gfxcourses.stanford.edu/cs149/fall22content/media/graphsdram/16_graphs.pdf

https://vercel.com/blog/the-rise-of-the-ai-crawler
-https://dadalo.pl/en/tech/gptbot-scans-internet-changing-content-consumption-future-of-search/